Laboratório Funcional · Aula 12

Eduardo Barella · RA 24088000-2 · Turma B
Gerado em 16/10/2026, 20:30:46 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora() é impura porque o resultado vem do relógio do sistema e não de um parâmetro, então cada vez que chama devolve uma coisa diferente.
poe(lista, x) é impura porque o append altera a lista que foi passada, ou seja, mexe em algo fora da função (efeito colateral).

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: A-1 A-2, porque o contador aumenta a cada chamada.
Linha 4: 14, já que g(4) = 12 e f(12) = 14.
Linha 7: acho que [7, 3, 9] [3, 7, 9], o sorted cria outra lista e a notas continua igual.

Saídas (original e com compor(g, f))

```
Original:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

```
Com compor(g, f):
4: 18
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque ela usa e altera a variável global contador, então o resultado depende de quantas vezes ela já foi chamada, e não só do "A". Isso atrapalha os testes porque o mesmo teste pode dar resultados diferentes dependendo da ordem em que as coisas rodaram, não dá pra testar a função sozinha.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(g(4)) = f(12) = 14
compor(g, f)(4) = g(f(4)) = g(6) = 18
A ordem muda o resultado: no primeiro multiplica por 3 e depois soma 2, no segundo soma 2 e depois multiplica por 3. A função de dentro é aplicada primeiro.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

O sorted(notas) devolve uma lista nova ordenada e deixa a original como estava. O notas.sort() ordena a própria lista e devolve None. O sorted é o estilo funcional, porque não altera nada, só recebe um valor e devolve outro.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
Original:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

Sem o apóstrofo, (car (A B C)):
22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
```

```
    191:35  3 (_ #f)
    196:27  2 (_ #f)
    223:20  1 (proc #<directory (guile-user) 14dd86230c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 14d…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (quote) diz pro Scheme não avaliar a lista, só tratar ela como dado. Sem ele, o Scheme tentou executar (A B C) como uma chamada de função, com A sendo a função. Como não existe nada chamado A, deu "Unbound variable: A".

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Acho que no Scheme funciona e imprime 1000000, porque a função é em cauda. No Python não tenho certeza, mas acho que vai dar erro, porque um milhão de chamadas deve passar do limite de recursão.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1) # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1) # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1) # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce a quantidade de multiplicações esperando (e a expressão fica cada vez maior), porque a conta n * ... só pode ser feita quando a chamada de dentro voltar. Na em cauda não cresce nada, porque o resultado parcial já vai no acumulador e não sobra nada pra fazer na volta.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

Porque o Scheme faz a otimização da chamada em cauda: como nada fica esperando, ele reaproveita o mesmo registro de ativação e a recursão funciona como um laço. O Python não faz isso, cada chamada empilha um registro novo, e ele tem um limite de profundidade (mais ou menos 1000). Por isso o 500 funcionou e o 1.000.000 deu RecursionError: maximum recursion depth exceeded.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3 ok
soma-quadrados: 14 ok
pares: (2 4 6) ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A mais difícil foi a pares, porque tem três casos e precisa montar uma lista nova com cons. Nas três funções o null? é o caso da lista vazia (devolve 0 ou '()) e o cdr é o que vai pra chamada recursiva. O car eu não usei na tamanho, porque ali só precisa contar mais 1. Na soma-quadrados uso o car pra elevar ao quadrado, e na pares uso o car no even? pra decidir se ele entra na lista ou não.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares acho que imprime [0,1,4,9,16], porque ele só pega os 5 primeiros. O primeiro 42 (div 1 0) fiquei na dúvida se ia dar erro de divisão por zero, mas como a função ignora o segundo parâmetro e o Haskell é preguiçoso, acho que imprime 42.

Saídas (original e com member 17)

```
Original:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Com member 17 squares (descomentado):
Time limit exceeded
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Por causa da avaliação preguiçosa: o Haskell só calcula os elementos quando precisa deles, então o take 5 faz ele calcular só os 5 primeiros da lista infinita. E o div 1 0 não deu erro porque a função primeiro nunca usa o segundo parâmetro, então ele nunca chega a ser calculado.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

Ficou rodando até o OneCompiler cortar com "Time limit exceeded". O 16 é um quadrado, então o member achou e parou. O 17 não é quadrado e a lista nunca acaba, então ele fica procurando pra sempre.
Pra corrigir, como squares é crescente, dá pra parar quando passar do número procurado (igual o member2 da p. 670):

```
member2 n (m:x)
 | m < n    = member2 n x
 | m == n   = True
 | otherwise = False
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) deve dar 15. E o map acho que dá [5, 10, 15], porque vezes(5) vira uma função que multiplica por 5.

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Porque a função devolvida é um fechamento: ela guarda a ligação com o ambiente onde foi criada, incluindo o n = 10. Então mesmo depois do somador terminar, o n continua existindo enquanto o mais10 existir.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Porque a ordem é diferente. O compose aplica o g primeiro: f(g(4)) = f(12) = 14. O andThen aplica o f primeiro: g(f(4)) = g(6) = 18. O que é igual ao compor(f, g) da Prática 1 é o f.compose(g), os dois deram 14.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
Exercício 1 (Java):
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
Exercício 2 (JavaScript):
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];
const aprovados = alunos
  .filter((a) ⇒ a.nota ⩾ 6)
  .map((a) ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
Exercício 1 (Java): 244
Exercício 2 (JavaScript): ANA, CARLA, DAVI
(mesmas saídas das versões originais com laço)
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A funcional é mais fácil de ler, porque já diz o que está sendo feito (filtra, transforma, soma) sem ter que acompanhar variável acumuladora e if dentro do for. Mas pra depurar acho a com laço mais fácil, porque dá pra colocar um print dentro do for e ver cada valor. No pipeline teria que quebrar a cadeia em partes pra ver os valores do meio.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A do Haskell é mais fácil de entender, porque em uma linha ela diz o que faz (soma dos cubos de 1 a n), parecido com a fórmula da matemática, sem variável de controle. Eu aceitaria ser mais lento num sistema comum de regras de negócio, onde é mais importante o código ser claro e ter menos bug. Não aceitaria num sistema embarcado ou de tempo real, onde desempenho é crítico.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi o Haskell conseguir usar uma lista infinita e não dar erro com div 1 0, só porque ele não calcula o que não usa.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro é no primeiro filtro: está b <- h, e o certo é b <= h. Com o <-, o Haskell entende que o b é tirado de dentro do h, como se h fosse uma lista.
Corrigido: sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]
Na mensagem, o compilador mostra que inferiu sort :: [a] -> [[a]], com h :: [a] e t :: [[a]], ou seja, achou que cada elemento era uma lista. Aí na linha 7 a lista passada pro sort tem elementos do tipo [a], mas o sort esperava elementos do tipo a, e dá o "Couldn't match expected type 'a' with actual type '[a]'".